DQN ASSIGNMENT

Deep Q-Network Control of the Unitree G1 Left Elbow

A Multi-Goal Reinforcement-Learning Assignment in MuJoCo and Gymnasium

CSCN8020 - Reinforcement Learning
Individual Assignment | 100 Marks
Prerequisite: Unitree MuJoCo G1 Primer Workshop completed

Assignment Goal

Train a student-written PyTorch DQN to control the Unitree G1 left elbow across multiple target angles, compare it with the existing rule-based policy, evaluate success rate, and demonstrate the learned policy visually.

1. Assignment Context

This assignment begins where the Unitree MuJoCo G1 Primer Workshop ends. Students have already inspected the raw model, implemented continuous elbow target control, validated the Gymnasium environment, and observed the live relationship among discrete actions, controller targets, actuator torques, and elbow movement.

Stage	Purpose
inspect_g1_model.py	Inspect the raw model, joint state, actuator limits, and viewer torque controls.
control_single_joint.py	Demonstrate continuous target-angle control using PD control and bias compensation.
test_g1_elbow_env.py --render	Validate the Gymnasium episode and success logic.
demo_g1_elbow_env.py	Prepare the camera and observe live actions, targets, torque values, and physical movement.
DQN	Replace the hand-written rule-based policy with a learned action-value policy.

The assignment does not require students to redesign the MuJoCo model or the Gymnasium environment. The approved environment is the controlled experimental platform. The student task is to implement, train, evaluate, and explain a DQN agent that learns the three-action decision policy.

2. Learning Objectives

1. Explain why DQN is suitable for the environment's discrete three-action control problem.

2. Map the four-value observation vector to three action-value estimates.

3. Implement an online Q-network and a target Q-network in PyTorch.

4. Implement experience replay and mini-batch learning.

5. Apply epsilon-greedy exploration with a controlled decay schedule.

6. Compute Bellman targets and optimize a temporal-difference loss.

7. Handle Gymnasium terminated and truncated signals correctly.

8. Train the agent headlessly and keep the implementation compatible with CPU execution.

9. Save and reload trained model checkpoints.

10. Evaluate the learned policy with exploration disabled.

11. Measure success rate separately from cumulative reward.

12. Compare the learned DQN with the provided rule-based baseline.

13. Interpret training curves and explain the effect of exploration decay.

14. Demonstrate the learned policy in the MuJoCo viewer after training.

3. Reinforcement-Learning Task

Objective. Train one DQN agent to move the fixed-base Unitree G1 left elbow to goals sampled from a limited multi-goal range and keep the elbow within the success tolerance for the required number of consecutive environment steps.

3.1 Environment

- Environment class: G1ElbowTargetEnv
- Controlled joint: left_elbow_joint
- Controlled actuator: left_elbow
- Default execution mode: headless
- Optional rendering: evaluation and demonstration only
- Low-level control: approved PD controller plus MuJoCo qfrc_bias compensation

3.2 Observation

[current elbow angle, current elbow velocity, goal angle, goal angle - current elbow angle]

3.3 Discrete Actions

Action	Meaning
0	Decrease the internal controller target
1	Hold the internal controller target
2	Increase the internal controller target

3.4 Multi-Goal Scope

During training, goal angles must be sampled from the approved range:

[-0.8, +0.8] rad

The final evaluation must include all four required benchmark goals:

-0.8 rad, -0.4 rad, +0.4 rad, +0.8 rad

Each benchmark goal must be evaluated in five episodes, producing 20 total evaluation episodes.

3.5 Success Requirement

Required threshold: at least 80% success over the 20 final evaluation episodes. This is equivalent to at least 16 successful episodes.

Evaluation must use a greedy policy with epsilon set to 0.0. Training success alone does not satisfy this requirement.

4. Medium-Scaffolding Model

Students begin from the completed primer repository. The following components are supplied and must remain compatible with the assignment:

- The fixed-base G1 MuJoCo model and assets

- The G1ElbowTargetEnv Gymnasium environment
- The approved low-level PD and bias-compensation controller
- The rule-based validation policy
- Environment checking and visualization scripts
- The successful multi-step success condition and reward function

Students must implement the DQN components themselves. Required student-written components are:

- ReplayBuffer
- QNetwork
- epsilon-greedy select_action()
- optimize_model() or an equivalent optimization function
- target-network synchronization
- training loop
- evaluation loop
- checkpoint saving and loading
- metrics recording and plotting

5. Required Methodology

5.1 PyTorch Network

PyTorch is required. The minimum approved architecture is:

```
Input: 4 values
Hidden layer 1: 64 units, ReLU
Hidden layer 2: 64 units, ReLU
Output: 3 Q-values
```

Students may use a larger network only if they justify the change and remain within the five-hour training limit. The final layer must not apply softmax because DQN outputs unconstrained Q-values.

5.2 Required DQN Elements

1. Create an online Q-network and a separate target Q-network.

2. Initialize the target network from the online network.

3. Store transitions in a replay buffer.

4. Do not optimize until the replay buffer contains enough samples for one mini-batch.

5. Select actions using epsilon-greedy exploration during training.

6. Sample random mini-batches from replay memory.

7. Calculate the current Q-value for each selected action.

8. Calculate the target using the reward, discount factor, target network, and next state.

9. Prevent bootstrapping from true terminated states.

10. Handle time-limit truncation explicitly and explain the chosen treatment in the report.

11. Optimize a Huber loss or mean-squared-error loss.

12. Clip gradients if necessary for stability.

13. Update the target network at the required interval.

14. Decay epsilon after each episode, subject to epsilon_min.

15. Save the best or final trained model.

5.3 Required Baseline Hyperparameters

Parameter	Required baseline value
Discount factor, gamma	0.95
Learning rate	0.001
Mini-batch size	64
Replay-buffer capacity	50,000 transitions
Initial epsilon	1.00
Minimum epsilon	0.05
Baseline epsilon decay	0.995 per episode
Target-network update	Every 250 optimization steps
Warm-up before learning	At least 500 transitions
Maximum episode length	Environment default: 150 steps
Training goal range	[-0.8, +0.8] rad
Evaluation epsilon	0.00

5.4 Training-Time Limit

The complete required training and comparison must finish within five hours on a student laptop. The implementation must run on CPU. CUDA may be detected and used optionally, but the submitted code must not require a GPU.

- Run training headlessly.
- Record wall-clock training time for each experiment.
- Use an episode cap or time-based stop condition.
- Do not render during training.
- Evaluation and the final video may use rendering after training is complete.

6. Required Parameter Study: Exploration Decay

Conduct one controlled comparison. Keep every other hyperparameter and random-seed policy unchanged.

Configuration	Epsilon decay	Purpose
A - Baseline	0.995	Longer exploration period
B - Faster decay	0.985	Earlier transition toward exploitation

For both configurations, report:

- Total training episodes
- Wall-clock training time
- Final epsilon
- Mean cumulative reward over the final 20 training episodes
- Training success rate over the final 50 episodes
- Final greedy evaluation success rate over 20 episodes
- Mean evaluation reward
- Observations about stability, convergence, and action behaviour

Recommendation. Select the better exploration-decay setting using evidence, not only the highest single reward. Consider success rate, stability, training time, and consistency across target angles.

7. Comparison with the Rule-Based Baseline

The existing choose_rule_based_action() implementation is the comparison baseline. Evaluate both the rule-based policy and the selected DQN policy on the same 20 benchmark episodes.

Metric	Rule-based policy	Selected DQN
Successes / 20		
Success rate		
Mean cumulative reward		
Mean episode length		
Mean final absolute error		
Main qualitative behaviour		

The discussion must address:

- Which policy is more sample efficient?
- Which policy is more stable near the goal?
- Does the DQN generalize across all four target angles?
- Does the DQN learn to use HOLD appropriately?
- Are there signs of oscillation or unnecessary target changes?
- Why might a hand-written policy outperform a learned policy in this simple task?

8. Required Experimental Workflow

1. Run the existing environment checker and rule-based baseline before training.

2. Set and record all random seeds used by Python, NumPy, PyTorch, and the Gymnasium environment.

3. Implement and compile the DQN source files.

4. Run a short smoke test to confirm replay insertion, batch sampling, action selection, and one optimization step.

5. Train Configuration A headlessly.

6. Train Configuration B headlessly.

7. Save model checkpoints and metrics for both configurations.

8. Evaluate both configurations greedily over 20 benchmark episodes.

9. Select the stronger DQN configuration using the required evidence.

10. Evaluate the rule-based baseline on the same benchmark goals.

11. Generate plots and the comparison table.

12. Record a short rendered video of the selected DQN after training.

13. Write the technical report and verify reproducibility from the submitted instructions.

9. Required Outputs and Metrics

9.1 Training Metrics

- Episode number
- Cumulative reward per episode
- Episode success indicator
- Episode length
- Final absolute angle error
- Epsilon value
- Loss summary when available
- Training wall-clock time

9.2 Required Plots

- Raw and moving-average training reward
- Training success rate using a rolling window
- Epsilon over episodes
- Loss over optimization steps or episodes
- Comparison plot or table for the two epsilon-decay configurations
- Evaluation success rate by target angle

9.3 Final Evaluation Table

Goal	Episodes	Successes	Success rate	Mean reward
-0.8 rad	5			
-0.4 rad	5			
+0.4 rad	5			
+0.8 rad	5			
Overall	20			

10. Deliverables

Deliverable	Required content
Python source code	Complete student-written DQN implementation and any approved starter files that were modified.
Trained model	Saved PyTorch checkpoint for the selected configuration.
Metrics files	CSV or equivalent structured output for both parameter configurations and final evaluation.
Plots	All required training, exploration, loss, success, and evaluation visualizations.
Technical report	A concise academic report, recommended length 6-10 pages excluding appendices.
Rendered video	A short 2-3 minute demonstration of the selected trained policy after headless training.
README update	Exact commands for training, evaluation, checkpoint loading, and rendering.

10.1 Recommended File Structure

```
src/
  g1_rl/
    g1_elbow_env.py
  dqn/
    __init__.py
    q_network.py
    replay_buffer.py
    agent.py
  train_dqn.py
  evaluate_dqn.py
  render_dqn_policy.py
results/
  config_a/
  config_b/
models/
  selected_dqn.pt
report/
  DQN_Assignment_Report.pdf
README.md
```

10.2 Video Requirements

- Length: approximately 2-3 minutes
- Load the saved model rather than retraining during the video
- Run with epsilon = 0.0
- Show at least two different target angles
- Show the MuJoCo viewer and relevant console metrics
- State whether each episode succeeded
- Do not use the rule-based policy in place of the DQN

11. Technical Report Requirements

1. Introduction and connection to the G1 Primer Workshop

2. Environment observation, action, reward, and success definitions

3. Final Q-network architecture

4. Replay-buffer and target-network methodology

5. Bellman target and loss formulation

6. Exploration strategy

7. Training methodology and reproducibility controls

8. Results for both epsilon-decay configurations

9. Required plots and evaluation tables

10. Comparison with the rule-based baseline

11. Discussion of failures, oscillation, stability, and generalization

12. Evidence-based recommendation of the better exploration-decay setting

13. Limitations and proposed future improvements

12. Technical and Academic Constraints

- This is an individual assignment.
- PyTorch is required.
- The implementation must run on CPU; CUDA support is optional.
- Training must run headlessly.
- The approved Gymnasium observation, actions, reward, and success condition must not be changed unless written instructor approval is obtained.
- The Unitree files under external/unitree_mujoco must not be edited.
- Do not replace DQN with another algorithm or a library implementation that hides the required DQN components.
- Stable-Baselines3 or equivalent turnkey RL training libraries are not permitted for the required implementation.
- Students must be able to explain every submitted DQN component.
- Any AI-assisted code or writing must be disclosed according to course and institutional requirements.
- The submitted video must demonstrate the learned DQN policy, not the existing rule-based policy.

13. Rubric - 100 Marks

Criterion	Marks	Full-credit expectations
A. Environment understanding and baseline verification	8	Correctly validates the supplied environment and rule-based baseline; accurately explains observations, actions, targets, torque control, reward, termination, and truncation.
B. Q-network and PyTorch implementation	12	Correct 4-input/3-output network, appropriate activations, device handling, initialization, forward pass, and checkpoint support.
C. Replay buffer and transition handling	10	Correct storage, bounded capacity, random batch sampling, tensor conversion, and handling of terminal information.
D. DQN action selection and exploration	10	Correct epsilon-greedy behaviour, decay, minimum epsilon, deterministic greedy evaluation, and reproducible seeds.

E. Bellman update and optimization	15	Correct selected Q-values, target-network bootstrap, terminal masking, detached targets, loss, optimizer use, and target-network update.
F. Training workflow and reproducibility	10	Headless training, CPU compatibility, time limit, metrics logging, checkpointing, clear commands, and repeatable execution.
G. Required exploration-decay comparison	10	Both configurations completed under controlled conditions; valid metrics and evidence-based comparison.
H. Final evaluation and performance	10	Correct 20-episode greedy evaluation across four goals. At least 80% success earns full performance credit; lower results are graded proportionally with consideration of correct methodology.
I. Rule-based versus DQN analysis	5	Fair comparison using common goals and metrics; insightful discussion of sample efficiency, stability, and generalization.
J. Report, plots, and interpretation	7	Complete academic report with readable plots, correct tables, technical interpretation, limitations, and recommendation.
K. Rendered video and submission quality	3	Clear 2-3 minute DQN demonstration, saved-model loading, multiple goals, organized files, and usable README.

13.1 Performance Threshold Interpretation

The 80% threshold is a required target, but grading will distinguish between an incorrect implementation and a correctly implemented agent that underperforms. The evaluation criterion will consider both achieved success and methodological correctness.

Final evaluation result	Performance interpretation
80-100% success	Meets or exceeds the required target
60-79% success	Partial performance credit; analysis of limitations required
1-59% success	Limited performance credit; implementation and diagnosis examined closely
0% or invalid evaluation	No performance credit; other rubric categories may still earn marks if valid

14. Submission Checklist

☐ Environment checker passes.

☐ Rule-based baseline results are recorded.

☐ Student-written DQN components are present.

☐ Both epsilon-decay experiments are complete.

☐ Training time is within five hours.

☐ CPU execution is supported.

☐ Selected checkpoint loads successfully.

☐ Evaluation uses epsilon = 0.0.

☐ Twenty evaluation episodes are reported.

☐ Overall success rate is calculated correctly.

☐ Rule-based and DQN policies are compared.

☐ All required plots are included.

☐ The rendered video shows the trained DQN.

☐ README commands have been tested.

☐ The technical report and all required files are included.

15. Final Academic Focus

The purpose of this assignment is not merely to produce a high reward. Students must demonstrate that they understand how the DQN transforms an observed robot state and goal into action values, how replay and target networks stabilize learning, how exploration affects data collection, and how a learned policy compares with a transparent rule-based controller.

The assignment is complete only when the code, evidence, interpretation, and rendered demonstration collectively show that the submitted policy is a trained DQN operating in the approved Unitree G1 Gymnasium environment.

16. Submission Requirements

These requirements define the files, repository structure, access conditions, and Brightspace submission items that must accompany the technical work described in Sections 1-15.

16.1 Required GitHub Repository

Create a GitHub repository with the exact name:

```
CSCN8020_Assignment3
```

The public repository URL must follow this format:

```
https://github.com/<USRID>/CSCN8020_Assignment3
```

where **<USRID>** is your GitHub account ID.

You must also provide a cloneable URL ending in **.git**:

```
https://github.com/<USRID>/CSCN8020_Assignment3.git
```

16.2 Required Repository Contents

• The completed Jupyter Notebook.

• All embedded assets required by the Notebook.

• Generated images or plots needed to reproduce the report.

• Training and evaluation log files or structured metrics files.

• README.md.

• requirements.txt.

• .gitignore.

• Student-written Python source files, saved model checkpoint, and any configuration files required by Sections 4-10.

> The repository must be organized so the evaluator can identify the Notebook, source code, results, plots, checkpoint, video link, and report without searching through unrelated files.

16.3 README.md Requirements

The README.md must be short but complete enough for the evaluator to understand, install, and run the project. It must include:

• Assignment title.

• Student full name.

• Student ID.

• A short project summary.

• Instructions for creating or activating the Python environment.

• Exact commands for installing dependencies.

• Exact commands for running the Notebook.

• Exact commands for training, evaluating, loading the checkpoint, and rendering the selected policy.

• A list and short description of the major repository files.

• A short description of the student-written DQN implementation.

• The GitHub repository URL and cloneable .git URL.

• The Python version and operating environment used for the final validated run.

16.4 requirements.txt Requirements

The requirements.txt file must list the Python packages needed to reproduce the submitted development environment. Typical packages may include:

```
torch
gymnasium
numpy
pandas
matplotlib
jupyter
mujoco
```

Include any additional libraries used by the Notebook, training scripts, plotting workflow, or video-generation workflow. The evaluator must be able to clone the repository, install the requirements, and execute the documented workflow.

16.5 .gitignore Requirements

A .gitignore file is mandatory. It must exclude unnecessary files and folders, including at minimum:

```
.venv/
venv/
env/
__pycache__/
.ipynb_checkpoints/
*.pyc
.DS_Store
```

Do not upload virtual environments, package caches, temporary rendering folders, large runtime-generated directories, duplicate checkpoints, or unnecessary MuJoCo cache data.

> **Repository size and clone-time requirement:** The repository must clone and be ready for evaluation in under three minutes on a standard internet connection. If the repository is unnecessarily large because it contains a virtual environment, cache folders, repeated model files, or other runtime sources, the assignment may be marked as incomplete.

16.6 One-Page PDF Submission to Brightspace

In addition to the GitHub repository, submit a separate one-page PDF directly to Brightspace. The PDF must contain:

• Student full name.

• Student ID.

• Assignment title.

• A project summary of approximately 100 words.

• The GitHub repository URL.

• The cloneable .git URL.

The summary may be adapted from README.md. The PDF must be exactly one page.

16.7 Brightspace Submission Requirements

Submit the following in Brightspace:

• The one-page PDF file.

• The GitHub repository link.

• The cloneable .git URL.

• The short rendered evaluation video, or a clearly accessible link to it, according to the instructor's Brightspace instructions.

> The GitHub repository must be public or otherwise accessible to the instructor at the time of grading. If the instructor cannot access or clone the repository, or if required files are unavailable, the assignment may be marked as incomplete.

16.8 Academic Integrity and AI-Use Statement

Students may use course materials, official documentation, textbooks, and AI tools to support learning and development. However, the submitted work must demonstrate the student's own understanding.

When AI tools are used, the student remains responsible for:

• Verifying the correctness of all submitted code.

• Understanding every part of the MuJoCo simulation and Gymnasium workflow.

• Explaining how the DQN algorithm works, including replay memory, epsilon-greedy exploration, Bellman targets, target networks, optimization, checkpointing, and evaluation.

• Ensuring that the Notebook and documented commands execute correctly.

• Ensuring that written explanations, equations, observations, and conclusions are accurate.

• Citing or acknowledging AI support where appropriate under course and institutional requirements.

• Providing original interpretation, testing evidence, observations, and conclusions.

> Submitting code, results, or explanations that the student cannot explain may be treated as an academic-integrity concern.

16.9 Final Submission Validation

Before submitting, complete the following final checks:

■ Clone the repository into a new temporary folder.

■ Create a fresh virtual environment.

■ Install requirements.txt.

■ Run the Notebook and required evaluation commands.

■ Confirm that the saved checkpoint loads without retraining.

■ Confirm that all plots, tables, logs, and links are present.

■ Confirm that the rule-based and DQN results in the report match the submitted metrics.

■ Confirm that the repository and .git URLs are correct and accessible.

■ Confirm that the Brightspace PDF is exactly one page.

■ Confirm that no virtual environment, cache, or unnecessary large directory is committed.

> The assignment is ready for grading only when an evaluator can clone the repository, install its documented dependencies, identify the required deliverables, load the trained checkpoint, reproduce the evaluation, and review the student's evidence without needing additional files or instructions.